

D7047E LAB3

Adesijibomi Aderinto

Oghenero Jeremiah Opia

2025-05-28

Concatenation for combining embeddings

Pros

- Retains full information from each embedding source.
- Simple to implement and easy to understand.
- Supports combining different types of embeddings.
- Provides richer and more expressive representations.
- Useful for merging multimodal features like text and images.

Cons

- Greatly increases the dimensionality of the input.
- Doesn't model interactions between embeddings directly.
- May introduce redundancy if embeddings are similar.
- Can lead to overfitting without enough data.
- Requires more computational resources and memory.

Addition for combining embeddings

Pros

- Keeps the combined embedding size the same as the original.
- Simple and efficient to implement with low computational cost.
- Reduces memory usage compared to concatenation.
- Encourages shared semantic space across embeddings.
- Helps in regularization by smoothing out noise in similar embeddings.

Cons

- May lose unique information from individual embeddings.
- Assumes embeddings are aligned in meaning and scale.
- Cannot capture complex interactions between sources.
- Risk of feature cancellation if vectors are opposing.
- Less expressive than learned or structured fusion methods

Multiplication for combining embeddings

Pros

- Captures interactions by emphasizing aligned dimensions.
- Keeps the embedding size the same, preserving memory.
- Can act as a form of soft feature selection.
- Suppresses noise by down-weighting uncorrelated features.
- Useful when embedding dimensions are semantically aligned.

Cons

- May zero out important features if one vector has small values.
- Assumes embeddings are on the same scale and aligned.
- Can lose unique information from individual embeddings.
- Not intuitive for combining very different types of embeddings.
- Less flexible and expressive than attention or gating methods.

Attention for combining embeddings

Pros

- Learns to focus on the most relevant parts of each embedding.
- Dynamically weights features based on context.
- Captures complex interactions between embeddings.
- Works well for combining multiple modalities or sequence data.
- Highly effective in state-of-the-art models like transformers.

Cons

- Computationally more expensive than simple operations.
- Requires more parameters and training data.
- Adds architectural complexity and harder to debug.
- Sensitive to initialization and hyperparameter tuning.
- May overfit on small datasets if not regularized properly.

Difference for combining embeddings

Pros

- Highlights contrast between embeddings.
- Useful for tasks involving comparison (e.g., similarity, entailment).
- Keeps embedding size constant and efficient.
- Emphasizes relational or directional information.
- Can help model subtle distinctions between input features.

Cons

- Loses original feature content from individual embeddings.
- Sensitive to noise and scale mismatches.
- Less informative if embeddings are already similar.
- Cannot capture complementary or additive features.
- Not ideal for multimodal or heterogeneous embedding types.